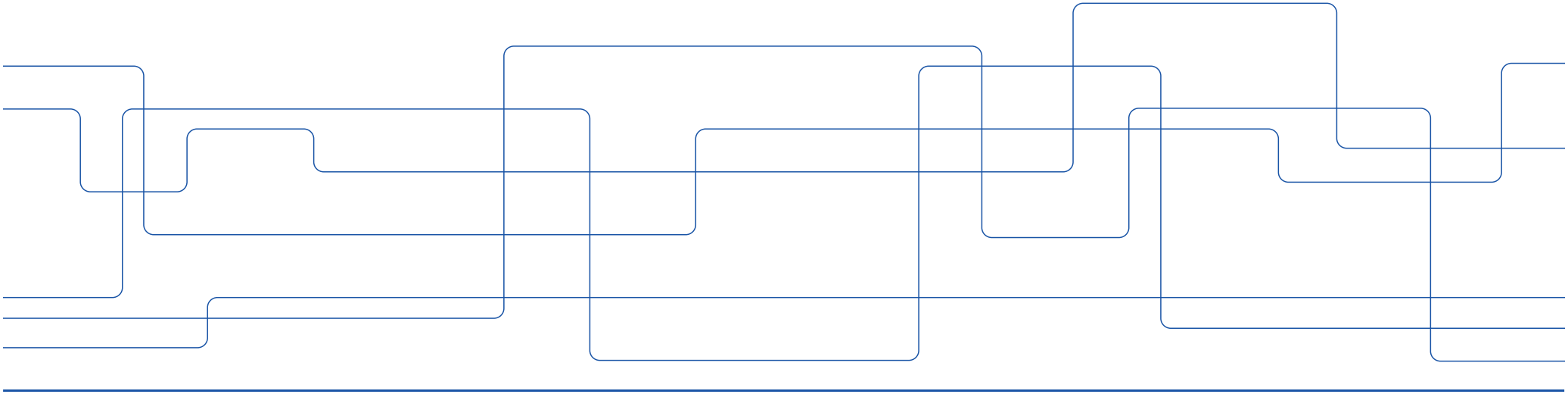


HI1024

Grundkurs i programmering

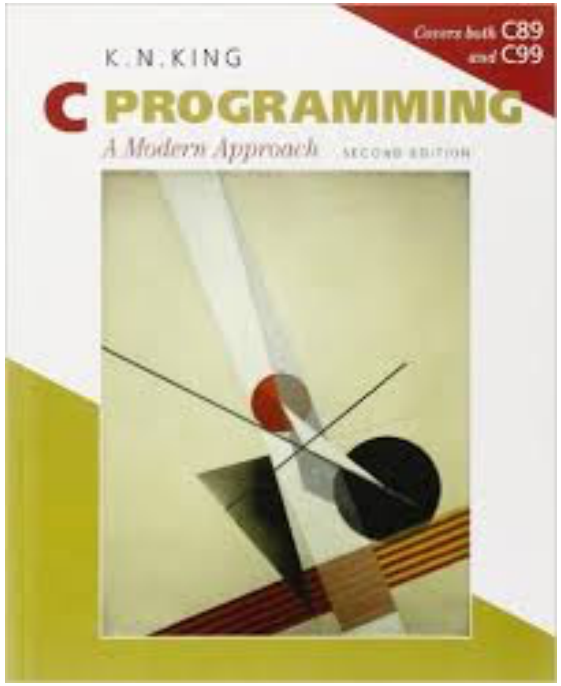
F2: Uttryck & operationer!

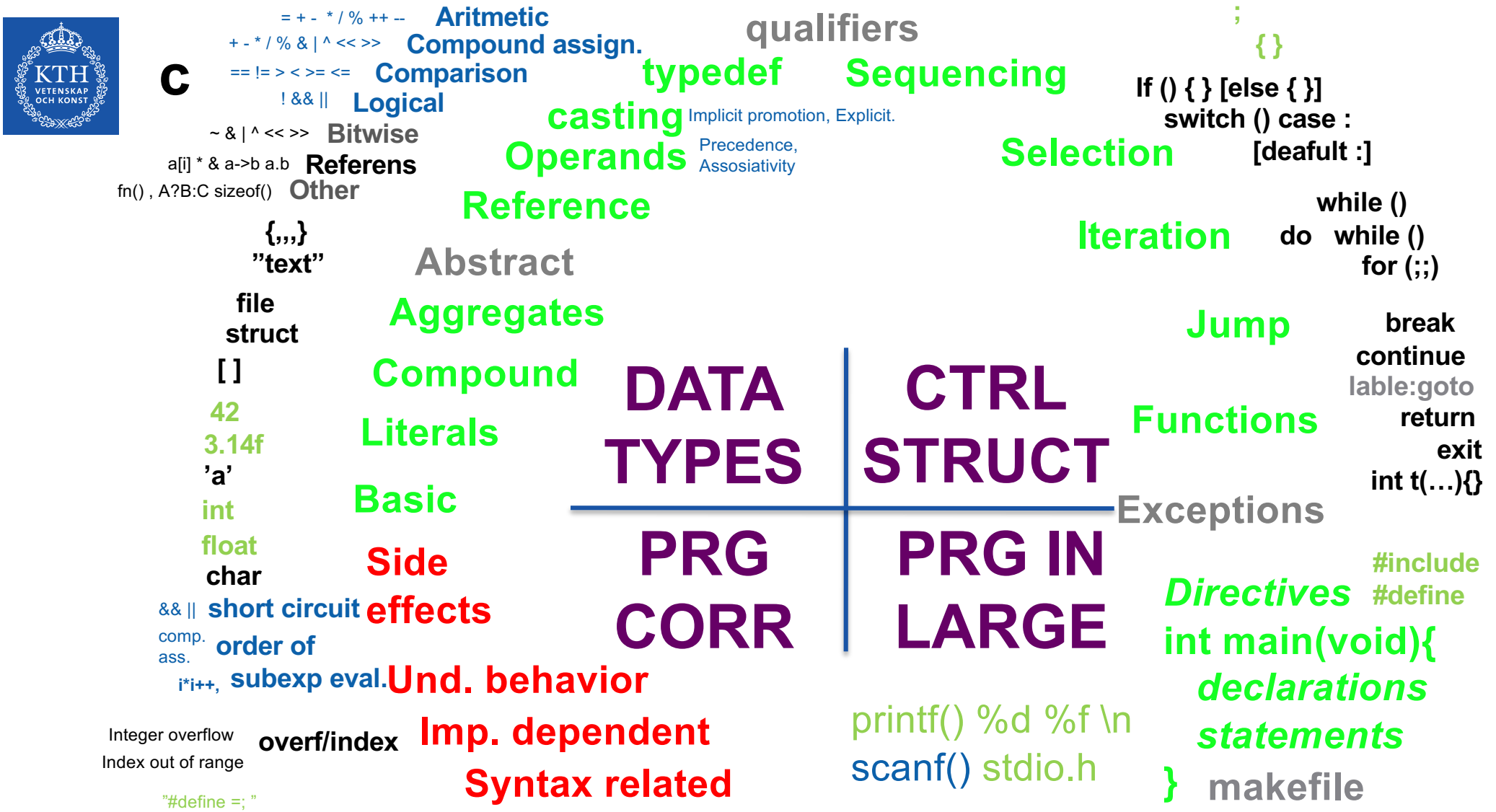


Innehåll!

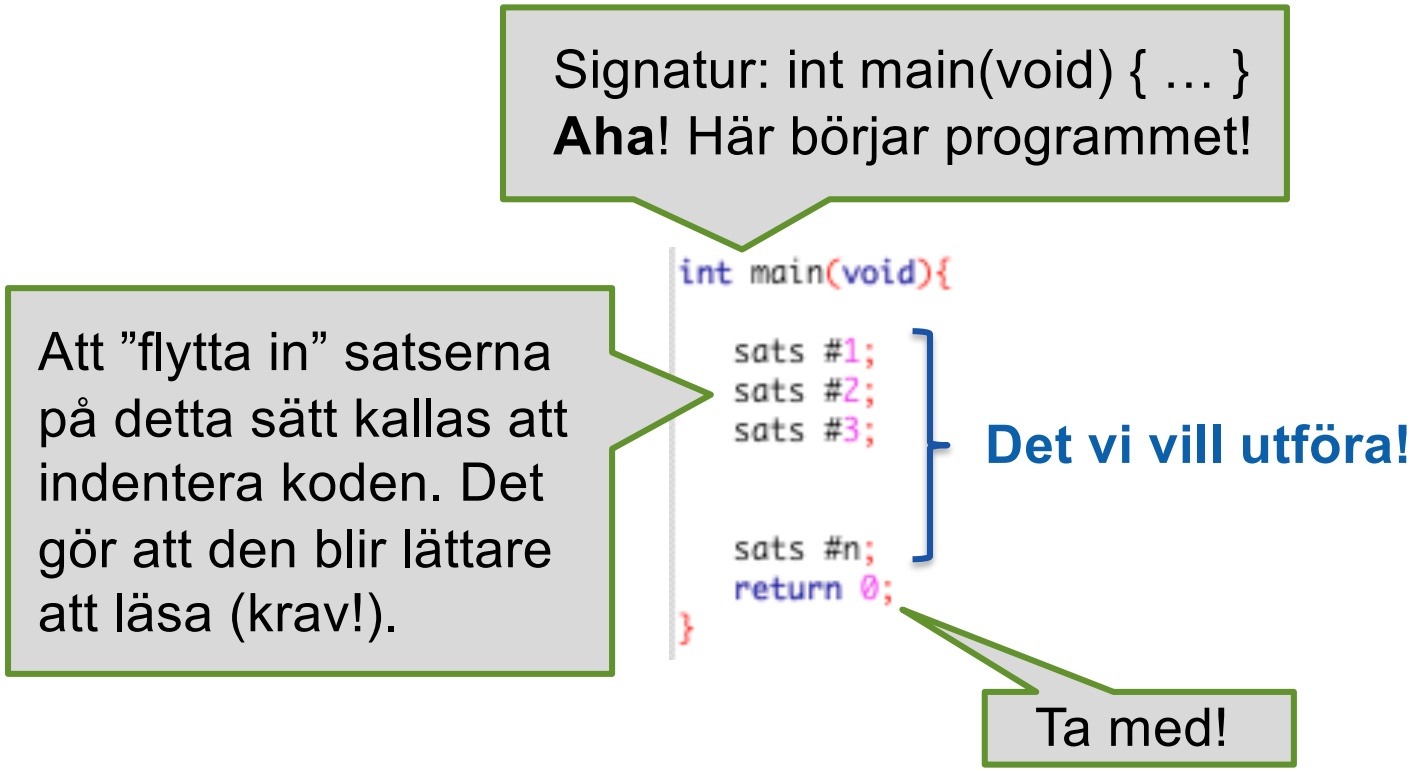
King 3.2, 4 & 5.1

- Repetition föreläsning #1
- scanf() i detalj
- Operationer & Uttryck
- Prioritet & association
- Övning

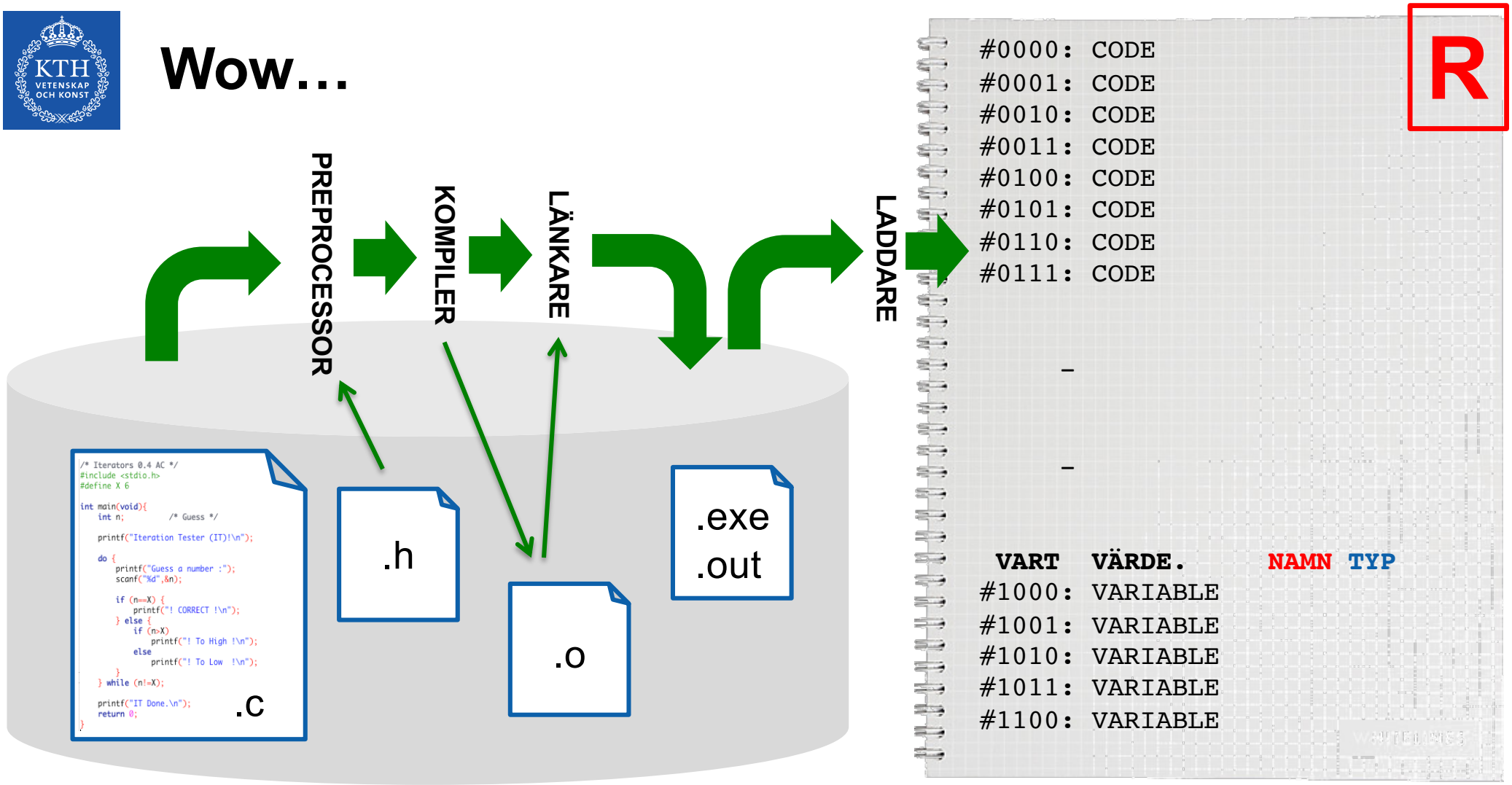



C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
Integer overflow
Index out of range
overf/index
”#define =; ”
printf() %d %f \n
scanf() stdio.h
int/int
/*comment*/ or //

Fram t.o.m Lab #1 ser alla program ut så här:



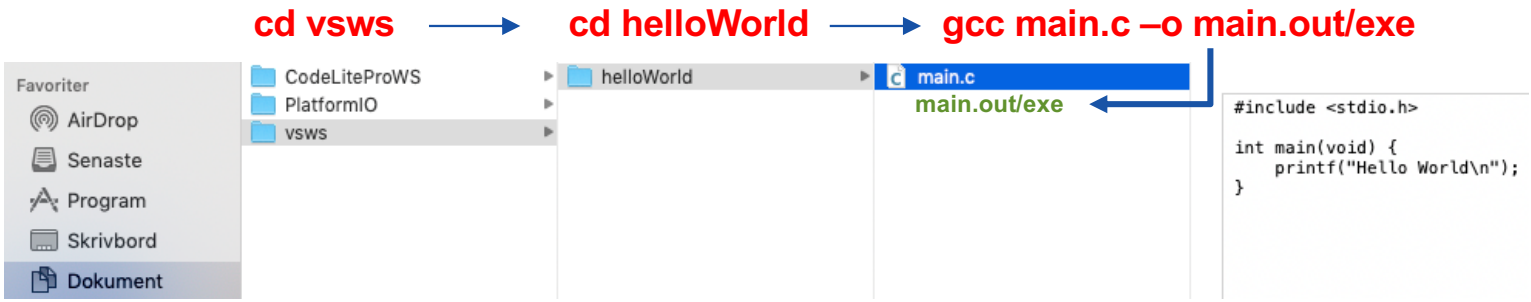
Wow…


PREPROCESSOR
KOMPILER
LÄNKARE
LADDARE
/* Iterators 0.4 AC */
#include <stdio.h>
#define X 6
int main(void){
int n; /* Guess */
printf("Iteration Tester (IT)!\n");
do {
printf("Guess a number :");
scanf("%d",&n);
if (n==X) {
printf("! CORRECT !\n");
} else {
if (n>X)
printf("! To High !\n");
else
printf("! To Low !\n");
}
} while (n!=X);
printf("IT Done.\n");
return 0;
}
.c
.h
.o
.exe
.out
#0000: CODE
#0001: CODE
#0010: CODE
#0011: CODE
#0100: CODE
#0101: CODE
#0110: CODE
#0111: CODE
VART VÄRDE. NAMN TYP
#1000: VARIABLE
#1001: VARIABLE
#1010: VARIABLE
#1011: VARIABLE
#1100: VARIABLE
R

Navigera, Kompilera & köra…

Du måste spara filen i editeringsfönster för att kunna kompilera!

Du måste navigera till rätt plats i filkatalogen, för att kompilera!



Du "kör" den skapade maskinkodsfilen med…

Mac: ./main.out

PC: .\main.exe



Kommentarer



Kommentarer är viktigt, fanns redan i Babage dator!

En kommentar skrivs mellan /* kommentar */

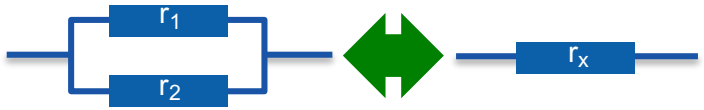
Kommentaren kan vara över flera rader, och ofta markerar utvecklingsmiljön kommentarsrader med en annan färg!

Kommentera alltid:

- **Vad är det för program.**
- **Variablers funktion**
- **Funktioners funktion**
- **Kluriga partier i koden!**

Resten av en rad kan göras till en kommentar med //! …// det här är en kommentar som slutar nu.

Vi testar det vi kan…





- Säg att vi vill utveckla ett program som kan beräkna den resistans som kan ersätta två (2) parallellkopplade motstånd…
- …formeln är enkel $r_x=(r_1*r_2)/(r_1+r_2)$, men vi gör som man alltid ska göra…
- …beskriver problemet i små, små, små steg så att ett barn kan förstå:

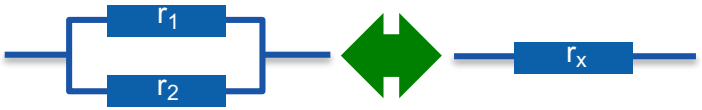
```
/* Starthälsning */

/* Fråga efter värdet på r1 */
/* Läs in värdet på r1 */
/* Fråga efter värdet på r2 */
/* Läs in värdet på r2 */
/* Beräkna rx */
/* Skriv ut svaret: rx! */

/* Avslutningshälsning */
```

1 **Skriv ner proble-met så att ett barn förstår det!**

Vi testar det vi kan…



R

```
/* Parallell Resistor Calcylator 0.1 AC */
#include <stdio.h>
int main(void){



    printf("Parallel Resistor Calcylator (PRC)\n");

    /* Fråga efter värdet på r1 */
    /* Läs in värdet på r1 */
    /* Fråga efter värdet på r2 */
    /* Läs in värdet på r2 */
    /* Beräkna rx */
    /* Skriv ut svaret: rx! */

    printf("PRC Done.\n");
    return 0;
}
```

2

Gör ett minimalt main() av dina anteckningar!

```
vcxxs-MacBook:~ vcxx$ /Users/vcxx/Desktop/main
Parallel Resistor Calcylator (PRC)
PRC Done.
vcxxs-MacBook:~ vcxx$
```

OK!

Vi testar det vi kan…

```c
/* Parallell Resistor Calcylator 0.2 AC */
#include <stdio.h>
int main(void){
    int r1 = 4;
    int r2 = 6;
    int rx;

    printf("Parallel Resistor Calcylator (PRC)\n");



    rx=(r1+r2)/(r1*r2);


    printf("PRC Done.\n");
    return 0;
}
```

```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.      NAMN  TYP
#1000:  4            r1   int
#1001:  6            r2   int
#1010:  ?            rx   int
#1011:  ?
#1100:  ?
```

R

Reservera plats i minnet för ett heltal!

```c
int r1 = 4;
```

Det här är en sats.

Kalla platsen för r1! Initiera med 4!

Vi testar det vi kan…

```
/* Parallell Resistor Calcylator 0.2 AC */
#include <stdio.h>
int main(void){
   int r1 = 4;
   int r2 = 6;
   int rx;

   printf("Parallel Resistor Calcylator (PRC)\n");



   rx=(r1+r2)/(r1*r2);


   printf("PRC Done.\n");
   return 0;
}
```

Variablerna **deklareras**
= minne reserveras för dem!

```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
  VART   VÄRDE.        NAMN  TYP
#1000: 4               r1    int
#1001: 6               r2    int
#1010: ?               rx    int
#1011: ?
#1100: ?
```

R

En variabel **måste** deklareras innan den används…
…kompilatorn kan kontrollera det och varnar (error)!

En variabel **måste** få ett värde innan den används…
…kompilatorn kan inte säkert testa det, svåra fel!

Den här informationen finns normalt inte i minnet, utan enbart medan programmet översätts!

Vi testar det vi kan…

```
/* Parallell Resistor Calcylator 0.2 AC */
#include <stdio.h>
int main(void){
    int r1 = 4;
    int r2 = 6;
    int rx;

    printf("Parallel Resistor Calcylator (PRC)\n");



    rx=(r1+r2)/(r1*r2);


    printf("PRC Done.\n");
    return 0;
}
```

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.       NAMN  TYP
#1000: 4              r1    int
#1001: 6              r2    int
#1010: ~~?~~ 2        rx    int
#1011: ?
#1100: ?
```

R

```
vcxxs-MacBook:~ vcxx$ /Users/vcxx/Desktop/main
Parallel Resistor Calcylator (PRC)
PRC Done.
vcxxs-MacBook:~ vcxx$
```

OK!

Vi kör programmet, det utförs rad för rad, rx får rätt värde – men vi kan ännu inte se det…

Nytt, mer om printf()…

```
/* Parallell Resistor Calcylator 0.3 AC */
#include <stdio.h>
int main(void){

   printformat( ”Text, platshållare & styrtecken” , uttryck… );

   printf("Replace with a %d ohm resistor.\n",rx);

   rx=(r1*r2)/(r1+r2);
   printf("Replace with a %d ohm resistor.\n",rx);

   printf("PRC Done.\n");
   return 0;
}
```

Det här är en sats.

%d = ”Byt ut mig mot ett heltal, som finns i variabeln!”

(\n = ”Börja på en ny rad (new line)!”)

```
  PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.        NAMN  TYP
#1000:  4              r1    int
 1001:  6              r2    int
 1010:  ~~?~~ 2        rx    int
 1011:  ?
 1100:  ?
```

3a..n

Konstanter

Variabler med värde

Uttryck & resultat!

```
vcxxs-MacBook:~ vcxx$ /Users/vcxx/Desktop/main
Parallel Resistor Calcylator (PRC)
Replace with a 2 ohm resistor.
PRC Done.
vcxxs-MacBook:~ vcxx$
```

OK!

Wow, när vi kör programmet dyker en ny rad upp på skärmen med det rätta svaret!

Nytt, ny funktion scanf()

%d = ”Läs in ett heltal!”

scanf("%d",&r1); ← Det här är en sats.

Till variabeln som finns på den här platsen! &r1=vart=10_{10}, r1=innehållet=4

```
/* Parallell
#include <st
int main(voi
   int r1;
   int r2;
   int rx;

   printf("P

   printf("First resistor  (ohm): ");
   scanf("%d",&r1);
   printf("Second resistor (ohm): ");
   scanf("%d",&r2);
   rx=(r1*r2)/(r1+r2);
   printf("Replace with a %d ohm resistor.\n",rx);

   printf("PRC Done.\n");
   return 0;
}
```

(& = Adressoperator.)

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
   *
   *
 VART   VÄRDE.      NAMN TYP
#1000: ~~?~~ 4        r1   int
#1001: ~~?~~ 6        r2   int
#1010: ~~?~~ 2        rx   int
#1011: ?
#1100: ?
```

R

4 Läs in riktiga värden!

```
vcxxs-MacBook:~ vcxx$ /Users/vcxx/Desktop/main
Parallel Resistor Calcylator (PRC)
First resistor  (ohm): 4
Second resistor (ohm): 6
Replace with a 2 ohm resisto
PRC Done.
vcxxs-MacBook:~ vcxx$
```

”Programmet” **stannar upp** här och väntar på att **du** matar in ett tal + retur!

OK!

Wow, det fungerar, och nu för godtyckliga värden på R1 & R2!

Övning #1: Reflektioner…

= ; , . π

```c
#include <stdio.h>
#define PI 3.1415
int main(int argc, char **argv)
{
    int r=10;
    printf("Sphere volume calc 1.0 AC\n");
    printf("Sphere r=%dcm have V=%-6.2fcm3\n",r,3*PI*r*r*r/4);
    printf("Program completed, bye!\n");
    return 0;
}
```

Liten bokstav variabelnamn
Stora bokstäver konstanter
Inga Å, Ä, Ö & mellanslag!

R³ ¾ 3.0/4

Övning #1: Reflektioner…

```c
#include <stdio.h>
#define PI 3.1415
int main(int argc, char **argv)
{
    int r=10;
    printf("Sphere volume calc 1
    printf("Sphere r=%dcm have \
    printf("Program completed, k
    return 0;
}
```

Various `main()` function declarations

1. `main()`
2. `main(int argc, char * argv[])`
3. `main(int argc, char ** argv)`
4. `int main()`
5. `int main(void)`
6. `int main(int argc, char * argv[])`
7. `int main(int argc, char ** argv)`
8. `int main(int argc, char * argv[], char *envp[])`
9. `int main(int argc, char ** argv, char ** envp)`
10. `void main()`
11. `void main(void)`
12. `void main(int argc, char * argv[])`
13. `void main(int argc, char ** argv)`

Det finns en handful korrekta sätt att namnge main-funktionen, vi använder de två markerade med grön pil!

Övning #1: Reflektioner…

```c
#include <stdio.h>
#define PI 3.1415
int main(int argc, char **argv)
{
    int r=10;
    printf("Sphere volume calc 1.0 AC\n");
    printf("Sphere r=%dcm have V=%-6.2fcm3\n",r,3*PI*r*r*r/4);
    printf("Program completed, bye!\n");
    return 0;
}
```

```c
#include <stdio.h>
#define PI 3.1314
int main(int argc, char **argv){int r=   10;printf("Sphere volume calc 1.0 AC\n"      );
printf(         "Sphere r=%dcm have V=%-6.2fcm3\n",      r,3*PI*r*r*r/     4);
printf("Program completed, bye!\n"); return 0;                                    }
```


…fungerar också, men det är nästan omöjligt att förstå vad koden gör, förbjudet i kursen!

Nytt: scanf() i detalj…

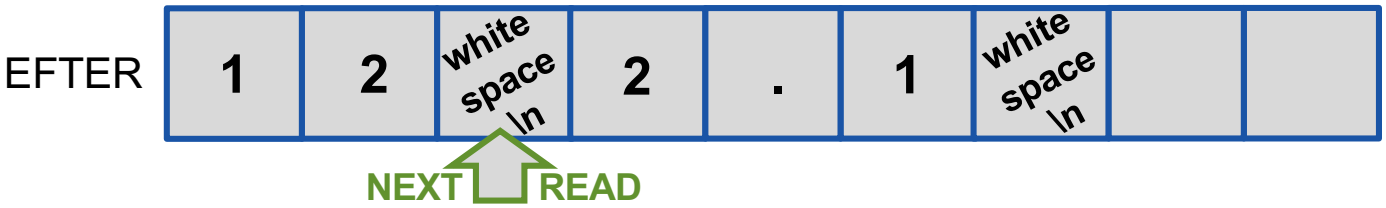






%d

1) Finns det några inledande ”mellanslag” så ignorera de! (Nej)
2) Om nästa tecken är en siffra, läs in den, tills det inte är en siffra! (1 2)

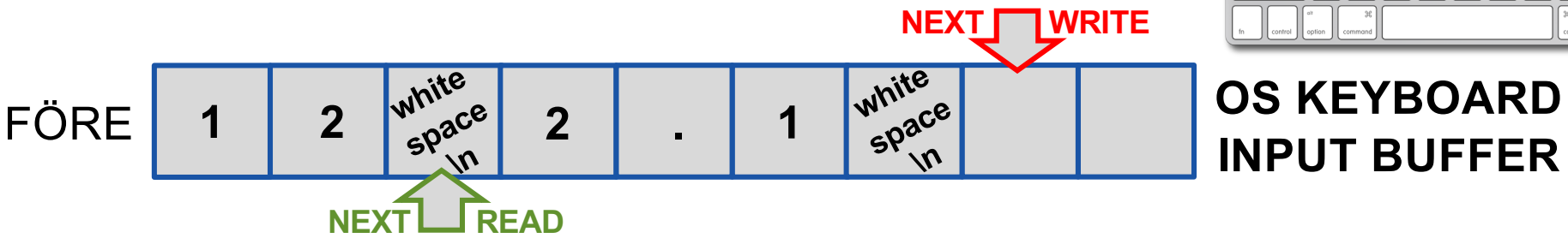


Wow, %d har läst in heltalet 12 från tgb!

Observera: trycket på return-tangenten finns kvar, men ”ser ut som” ett mellanslag!

Nytt: scanf() i detalj…

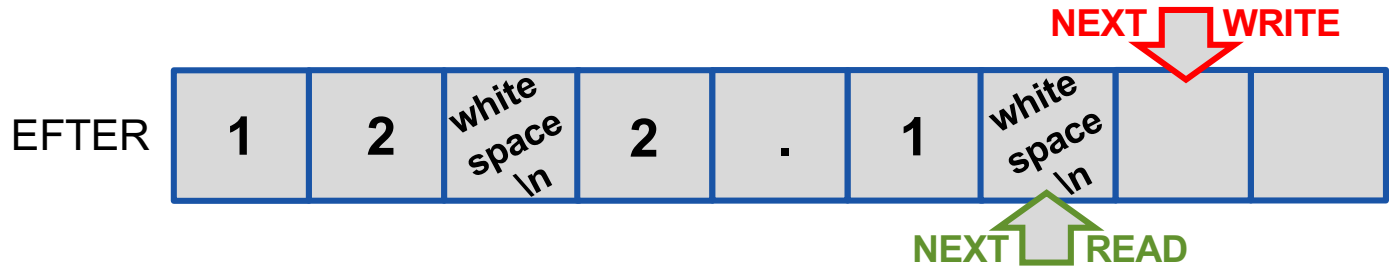




OS KEYBOARD INPUT BUFFER

%f

1) Finns det några inledande ”mellanslag” så ignorera de! (**Ja!**)
2) Om nästa tecken är en siffra, läs in den, tills det inte är en siffra! (2)
3) Är nästa tecken en punkt så läs in den och fortsätt… (Ja)
4) Om nästa tecken är en siffra, läs in den, tills det inte är en siffra! (1)



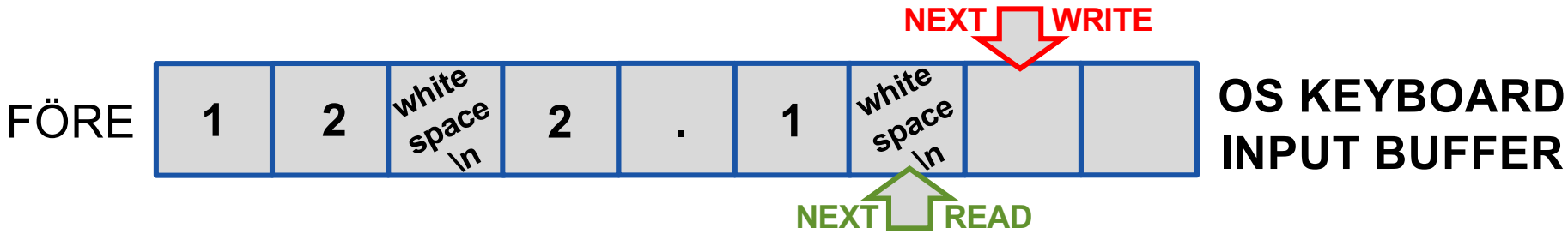
Wow, %f har läst in decimaltalet 2.1!

Observera: det finns fortfarande kvar ett ”mellanslag”!

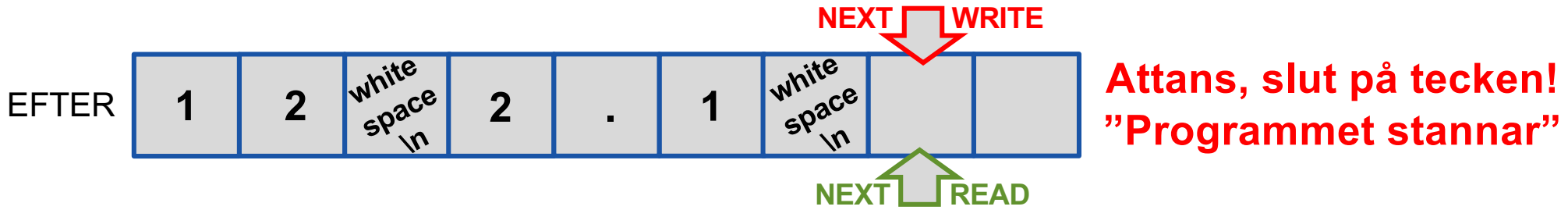
Nytt: scanf() i detalj…







%d 1) Finns det några inledande ”mellanslag” så ignorera de! (Ja)



2) Om nästa tecken är en siffra, läs in den, tills det inte är en siffra! (1 2)

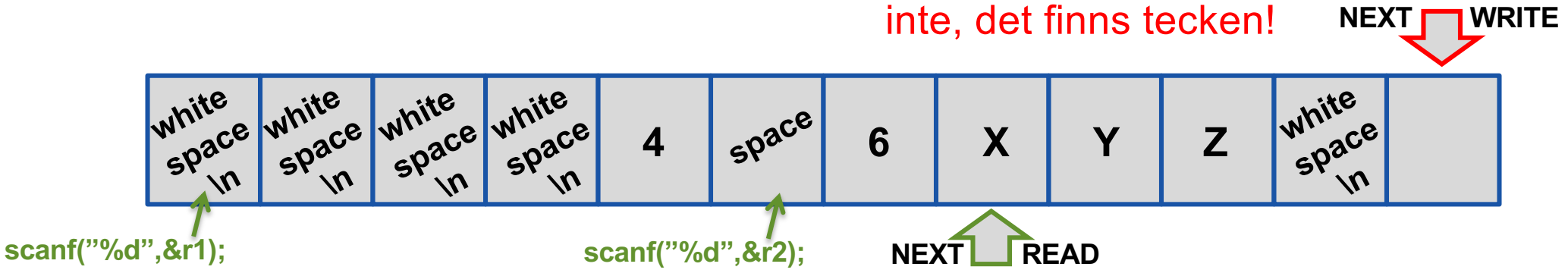
Nytt: scanf() i detalj…

```
vcxxs-macbook:~ vcxx$ /Users/vcxx/Desktop/main
Parallel Resistor Calcylator (PRC)
First resistor  (ohm):
```

OS Keyboard buffer är tom, ”programmet” stannar!, du matar in [4*retur4 6YZretur]

```
4 6XYZ
Second resistor (ohm): Replace with a 2 ohm resistor.
PRC Done.
vcxxs-macbook:~ vcxx$
```

OBS: Programmet stannar inte, det finns tecken!



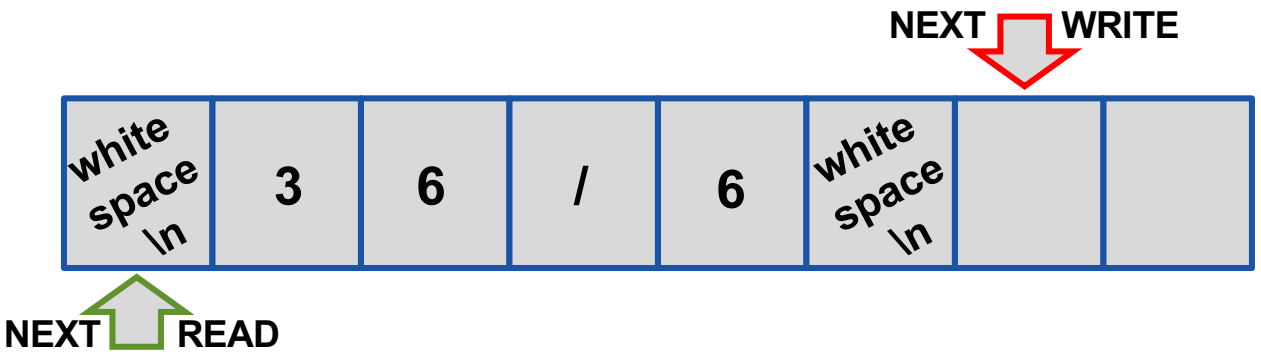
Väldigt vanligt fel, det ser ut som om den ”hoppar över” en inmatning!

Nytt: scanf() i detalj…

Mönsterpassning!

Hitta en %d…
…som måste följas av ett ’/’…
…som måste följas av ett %d!

scanf(”%d/%d”,&numerator, &denominator);



Blanda inte ihop scanf:s mönsterpassning med printf:s styrtecken (aldrig \n i scanf)!



Gruppuppgift!

Läs in ett personnummer på formatet YY.MM.DD och skriv ut år, månad och dag under varandra på separata rader!

```
SSNb format converter
Enter a SSNb (YY.MM.DD):62.05.09
Year  : 62
Month : 05
Day   : 09
SSNb format converter, bye.
```

1 Skriv ner problemet så att ett barn förstår det!

2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter
Variabler med
Värde Uttryck
a..n & resultat!

4 Läs in riktiga värden!



Gruppuppgift!

1 **Skriv ner proble-met så att ett barn förstår det!**

```
// Skriv ut välkomstmeddelande

// Förklara format på inmatning
// Läs in YY.MM.DD;

// Skriv ut year
// Skriv ut month
// Skriv ut day

// Skriv ut avslutningsmeddelande
```

Gruppuppgift!

1 Skriv ner proble-met så att ett barn förstår det!

2 Gör ett minimalt main() av dina anteckningar!

```c
#include <stdio.h>

int main(int argc, char **argv) {
    // int year, month, day
    printf("SSNb format converter\n");

    printf("Enter a SSNb (YY.MM.DD):\n");
    // Läs in YY.MM.DD;

    // Skriv ut year
    // Skriv ut day
    // Skriv ut day

    printf("SSNb format converter, bye.\n");
    return 0;
}
```

```
SSNb format converter
Enter a SSNb (YY.MM.DD):
SSNb format converter, bye.
```

Gruppuppgift!

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int year=62, month=9, day=5;
    printf("SSNb format converter\n");

    printf("Enter a SSNb (YY.MM.DD):");
    //Läs in YY.MM.DD;

    printf("Year  : %02d\n",year);
    printf("Month : %02d\n",month);
    printf("Day   : %02d\n",day);

    printf("SSNb format converter, bye.\n");
    return 0;
}
```

1 Skriv ner problemet så att ett barn förstår det!

2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter Variabler med Värde Uttryck & resultat!

a..n

```
SSNb format converter
Enter a SSNb (YY.MM.DD):
Year  : 62
Month : 09
Day   : 05
SSNb format converter, bye.
```

Gruppuppgift!

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int year=62, month=9, day=5;
    printf("SSNb format converter\n");

    printf("Enter a SSNb (YY.MM.DD):");
    scanf("%d.%d.%d",&year, &month, &day);

    printf("Year  : %02d\n",year);
    printf("Month : %02d\n",month);
    printf("Day   : %02d\n",day);

    printf("SSNb format converter, bye.\n");
    return 0;
}
```

```
SSNb format converter
Enter a SSNb (YY.MM.DD):62.05.09
Year  : 62
Month : 05
Day   : 09
SSNb format converter, bye.
```

1 Skriv ner problemet så att ett barn förstår det!

2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter Variabler med Värde Uttryck & resultat!

a..n

4 Läs in riktiga värden!



Operatorer och uttryck

```
rx=(r1*r2)/(r1+r2);
```

Vi har redan sätt att det går att skriva matematiska uttryck på förväntat sätt…

…precis som i matematiken finns det regler för:

- Vilken prioritet (precedence) olika operatorer (op) har, t.ex utför * före +
- Om en op hör till termen till höger eller vänster (associativity), =-b är =(-b)

Observera att ’=’ ger uttrycket värdet av högerledet, x=2; har värdet 2…

…som kan användas t.ex till en ny tilldelning y=x=2; ger att både x & y blir 2!



Operatorer och uttryck

```
rx=(r1*r2)/(r1+r2);
```

C-spåket utför i enkla fall implicit typkonvertering…

…om en int-variabel i tilldelas 2.4 (i=2.4f;) så trunkeras 2.4 till 2, så i blir 2!

Det här kan leda till mycket svårfunna fel…

…om vi också har en flyttalsvariabel f och skriver f=i=2.4; vad blir f & i?



Operatorer och uttryck

```
rx=(r1*r2)/(r1+r2);
```

C-spåket utför i enkla fall implicit typkonvertering…

…om en int-variabel i tilldelas 2.4 (i=2.4f;) så trunkeras 2.4 till 2, så i blir 2!

Det här kan leda till mycket svårfunna fel…

…om vi också har en flyttalsvariabel f och skriver f=i=2.4; vad blir f & i?

Först blir i=2 genom implicit typkonvertering…
…därefter genom en till implicit typkonverteing av resultatet blir f=2.0f!

Det går också att utföra explicit typkonvertering (cast) men det får vänta…

Det finns några stycken…

= + - * / % ++ -- **Aritmetic**

+ - * / % & | ^ << >> **Compound assign.**

== != > < >= <= **Comparison**

! && || **Logical**

~ & | ^ << >> **Bitwise**

a[i] * & a->b a.b **Referens**

fn() , A?B:C sizeof() **Other**

…vi tittar på några av dem…

…men viktigast är nog att aldrig glömma bort att **använda parenteser ()!**

Precedence	Operator	Description	Associativity
1	++ -- () [] . –> (type){list}	Suffix/postfix increment and decrement Function call Array subscripting Structure and union member access Structure and union member access through pointer Compound literal(C99)	Left-to-right
2	++ -- + – ! ~ (type) * & sizeof _Alignof	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT Type cast Indirection (dereference) Address-of Size-of Alignment requirement(C11)	Right-to-left
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ –	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <= > >=	For relational operators < and ≤ respectively For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10	|	Bitwise OR (inclusive or)	
11	&&	Logical AND	
12	||	Logical OR	
13[note 1]	?:	Ternary conditional[note 2]	Right-to-Left
14	= += –= *= /= %= <<= >>= &= ^= |=	Simple assignment Assignment by sum and difference Assignment by product, quotient, and remainder Assignment by bitwise left shift and right shift Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

Aritmetiska

```
int r1=6, r2=4, rx, ry;

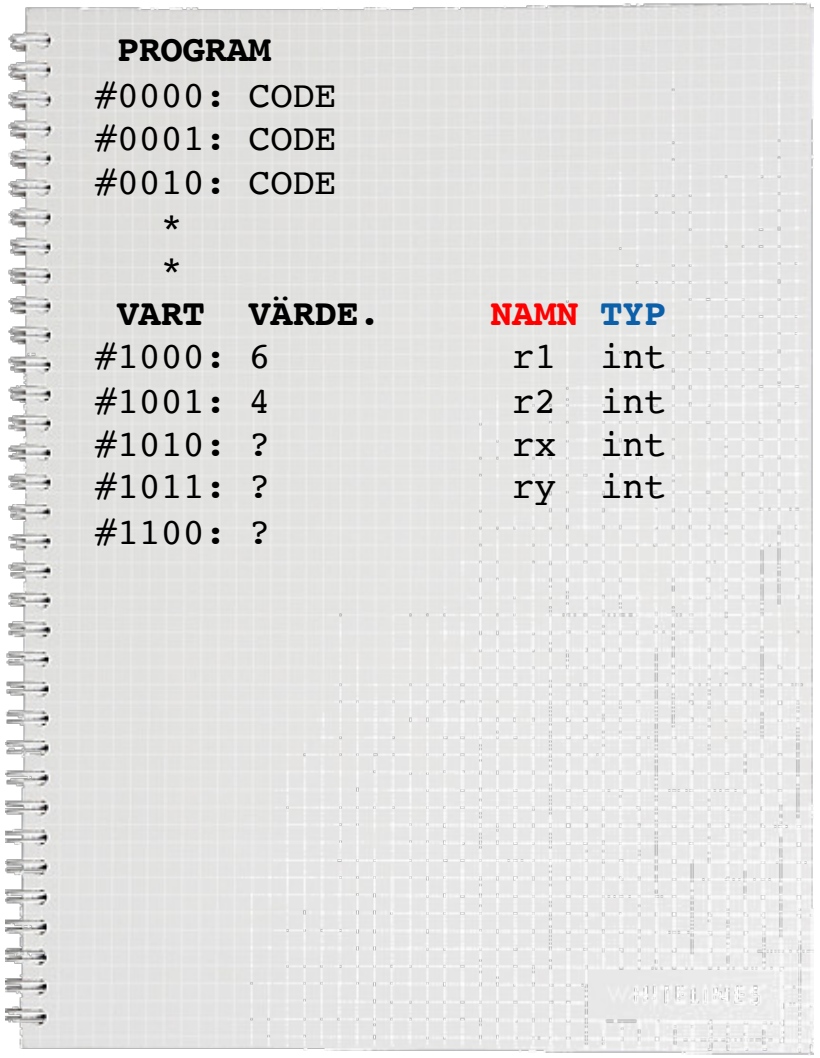
rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);
printf("%d\n", ++r1);

rx+=5;
```

Inget konstigt…
…ge mig 4 int variabler…
…2 av dem har startvärden.



Aritmetiska

```
int r1=6, r2=4, rx, ry;

rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);
printf("%d\n", ++r1);

rx+=5;
```

HL: int / int är int
HL: 6 / 4 är 1
VL: tilldelas 1!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.        NAMN  TYP
#1000: 6               r1   int
#1001: 4               r2   int
#1010: ~~?~~ 1         rx   int
#1011: ?               ry   int
#1100: ?
```

Aritmetiska

```
int r1=6, r2=4, rx, ry;

rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);
printf("%d\n", ++r1);

rx+=5;
```

HL: % är rest vid heltalsdivision
HL: 6 % 4 är 2
VL: tilldelas 2!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.        NAMN  TYP
#1000: 6               r1   int
#1001: 4               r2   int
#1010: ~~?~~ 1         rx   int
#1011: ~~?~~ 2         ry   int
#1100: ?
```


% är otroligt användbart, t.ex x%2 avslöjar om x är ett udda eller jämt tal!

Aritmetiska

```
int r1=6, r2=4, rx, ry;

rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);
printf("%d\n", ++r1);

rx+=5;
```

HL: vanlig addition
HL: 1 + 5 = 6
VL: tilldelas 6!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.        NAMN  TYP
#1000: 6               r1   int
#1001: 4               r2   int
#1010: ~~?~~ ~~1~~ 6   rx   int
#1011: ~~?~~ 2         ry   int
#1100: ?
```


Hämta rx ursprungliga värde (1), addera till 5, spara resultatet (6) som rx nya värde!

Aritmetiska

```
int r1=6, r2=4, rx, ry;

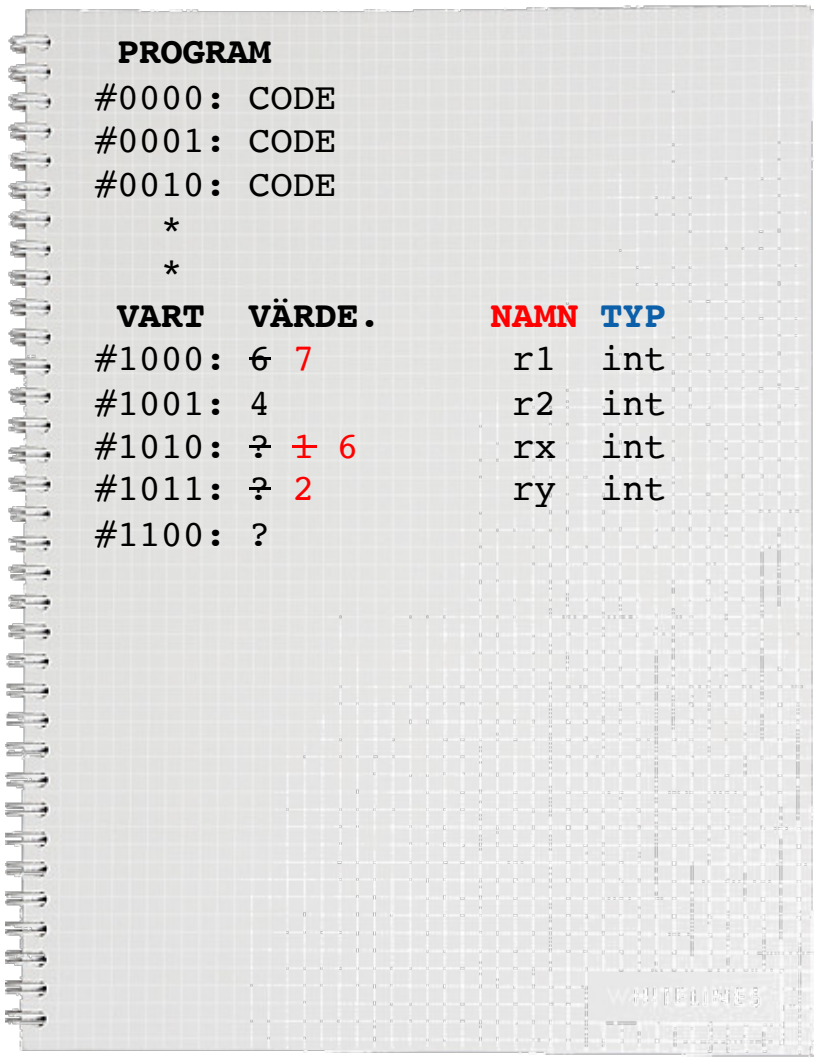
rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);   (Skriver ut 6!)
printf("%d\n", ++r1);

rx+=5;
```

++ betyder öka värdet med 1
Står ++ **efter** variablen så används ursprungsvärdet i uttrycket!



Hämta r1 ursprungliga värde (6), skriv ut 6, öka r1 med 1 till 7 när utskriften är klar!

Aritmetiska

```
int r1=6, r2=4, rx, ry;

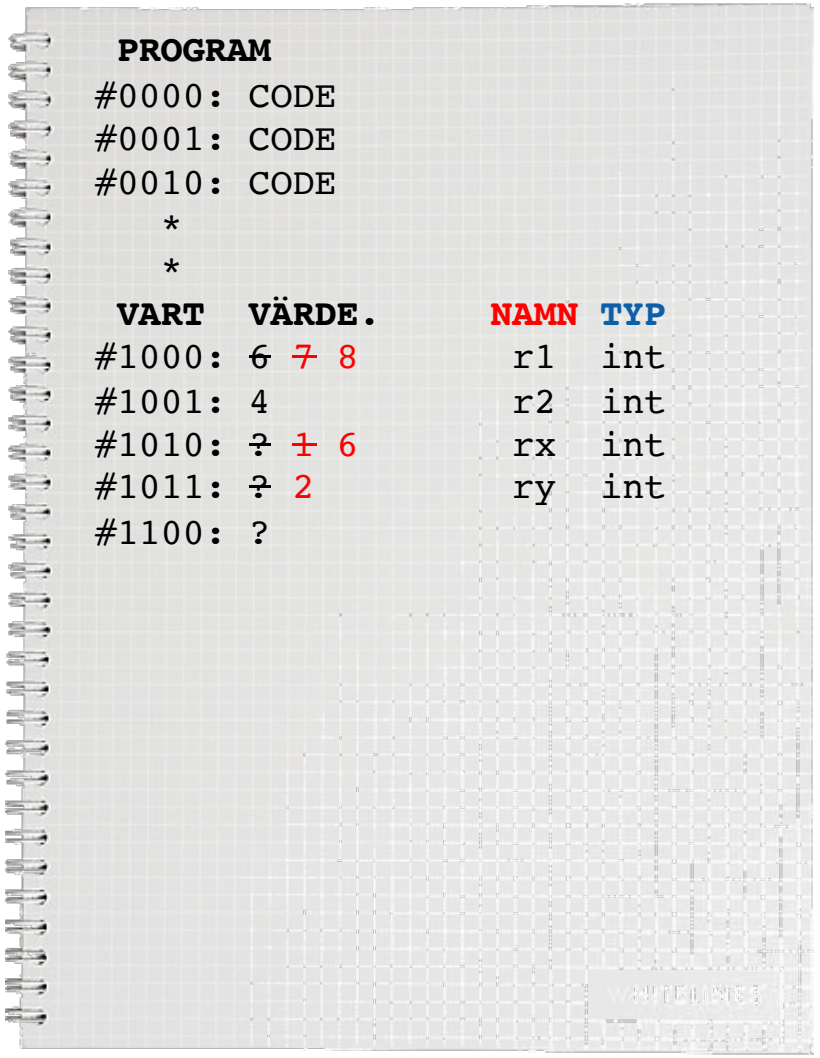
rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);   (Skriver ut 6!)
printf("%d\n", ++r1);   (Skriver ut 8!)

rx+=5;
```

++ betyder öka värdet med 1
Står ++ **före** variablen så används det uppdaterade värdet i uttrycket!



Hämta r1 ursprungliga värde (7), öka r1 med 1 till 8, använd det nya värdet för utskriften!

Aritmetiska

```
int r1=6, r2=4, rx, ry;

rx = r1 / r2;
ry = r1 % r2;

rx = rx + 5;

printf("%d\n", r1++);   (Skriver ut 6!)
printf("%d\n", ++r1);   (Skriver ut 8!)

rx+=5;
```

+= kortform för x = x + y…

…mao rx+=5 är rx=rx+5…

…undvik, kan leda till svåra fel!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.        NAMN  TYP
#1000: 6 7 8           r1   int
#1001: 4               r2   int
#1010: ? 1 6 11        rx   int
#1011: ? 2             ry   int
#1100: ?
```

(order of subexpression evaluation)

Du behöver i^2 och vill därefter öka i med 1 så du skriver i*i++, går men resultatet är odefinerat i c!

Jämförande

Sant eller falsk representeras i c-språket på följande (lite märkliga) sätt:

Falskt : Heltalet 0

Sant. : Godtyckligt heltal utom 0, med fördel 1!

Operator	Meaning of Operator	Example
==	Equal to	5==3 returns false (0)
>	Greater than	5>3 returns true (1)
<	Less than	5<3 returns false (0)
!=	Not equal to	5!=3 returns true(1)
>=	Greater than or equal to	5>=3 returns true (1)
<=	Less than or equal to	5<=3 return false (0)

”Resultatet” av en jämförelse kan med andra ord skrivas ut med %d, eller användas som en int där det är tillåtet! x=x+(5>3); blir x=x+1;!

(Se upp med: = är tilldelning, == är lika med, det kan bli otroligt fel ibland!)

Jämförande

```c
/* Logical Expressions 0.1 AC */
#include <stdio.h>

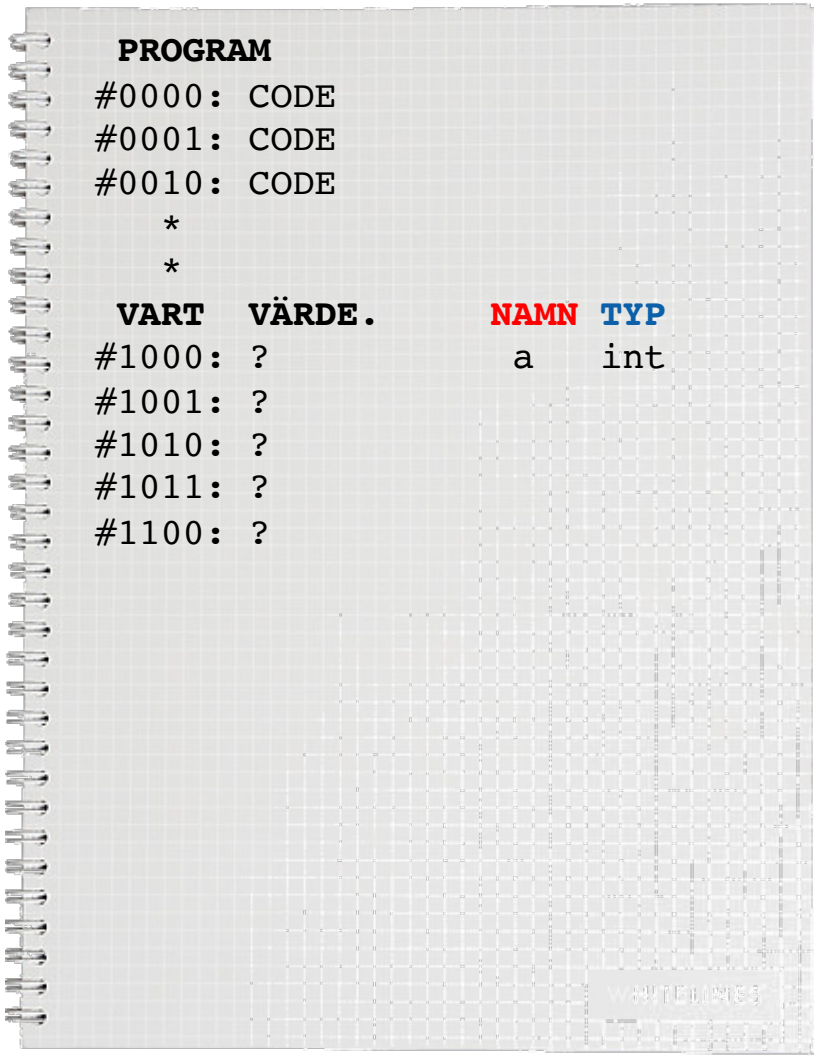
int main(void){
    int a;

    printf("Logical Expressions Tester (LET)!\n");

    printf("4==5 evaluates to: %d\n", 4==5);
    a = (4==4);
    printf("4==4 evaluates to: %d\n", a);

    printf("LET Done.\n");
    return 0;
}
```

Inget konstigt…
…ge mig en int variabler…
…utan startvärde.



Jämförande

```c
/* Logical Expressions 0.1 AC */
#include <stdio.h>

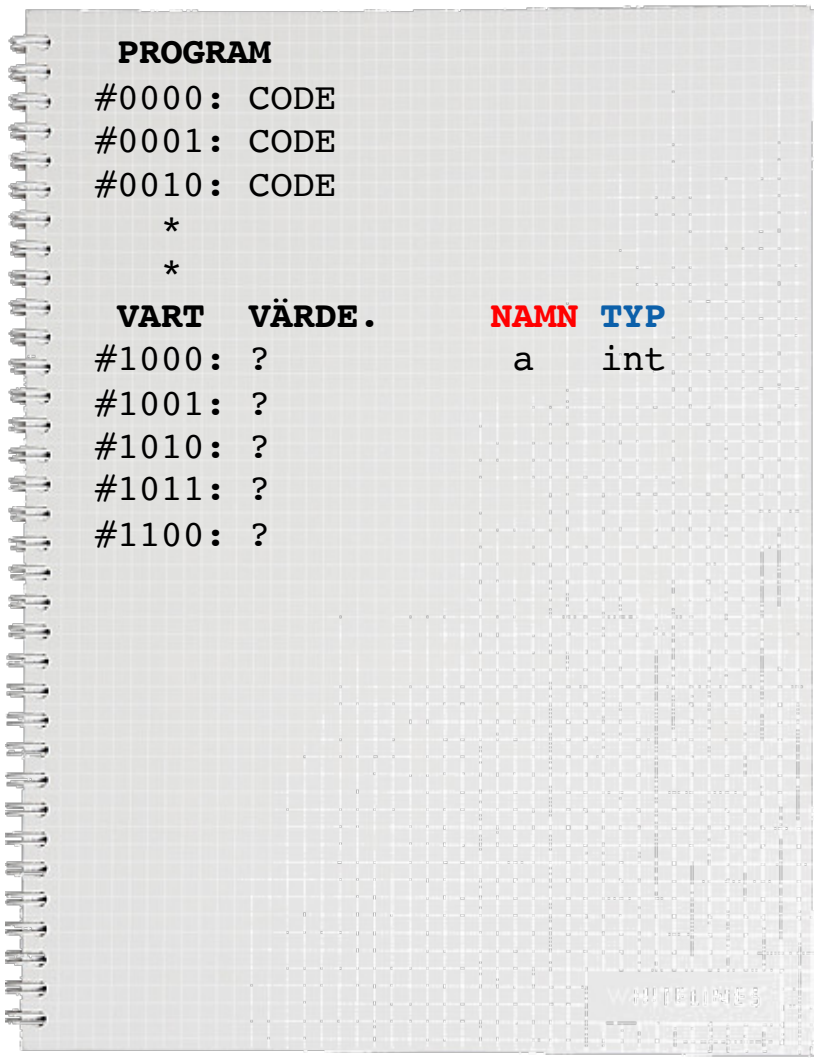
int main(void){
    int a;

    printf("Logical Expressions Tester (LET)!\n");

    printf("4==5 evaluates to: %d\n", 4==5);
    a = (4==4);
    printf("4==4 evaluates to: %d\n", a);

    printf("LET Done.\n");
    return 0;
}
```

Hum, ett jämförande uttryck…
…4 är inte lika med 5…
…så returnera falskt (0)!



Jämförande

```c
/* Logical Expressions 0.1 AC */
#include <stdio.h>

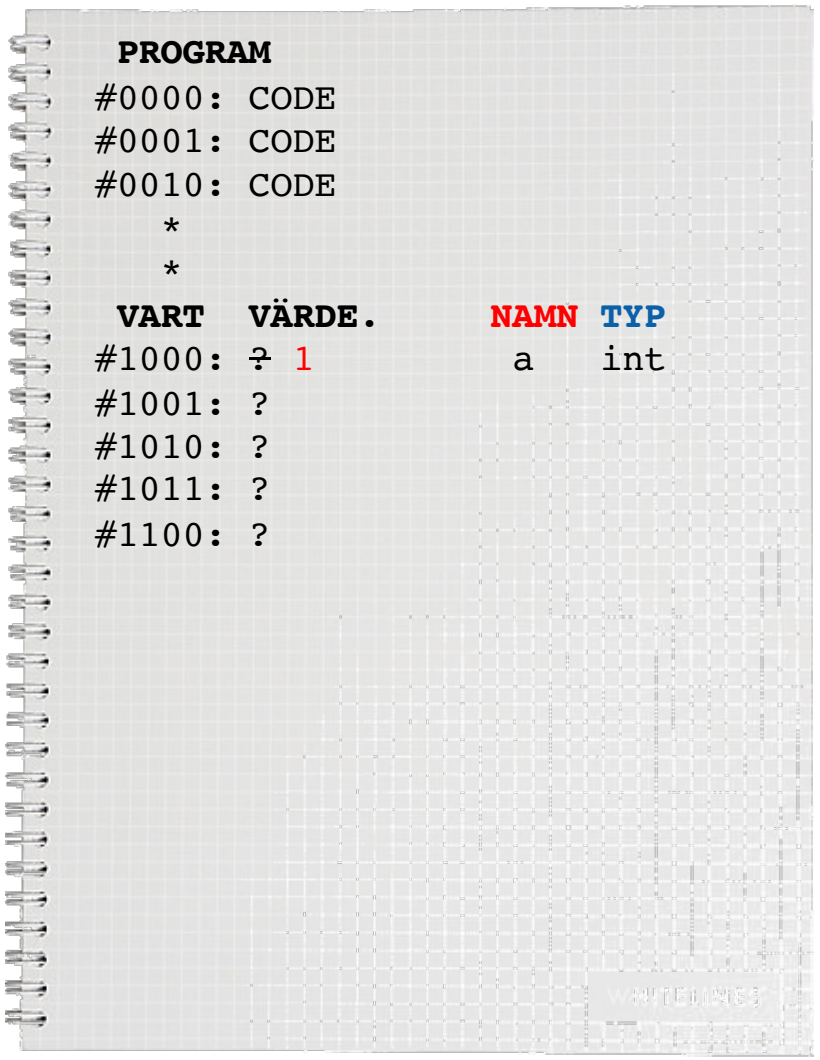
int main(void){
    int a;

    printf("Logical Expressions Tester (LET)!\n");

    printf("4==5 evaluates to: %d\n", 4==5);
    a = (4==4);
    printf("4==4 evaluates to: %d\n", a);

    printf("LET Done.\n");
    return 0;
}
```

HL: ett jämförande uttryck…
HL: 4 är lika med 4, det är sant (1)…
VL: tilldela a = 1!



Jämförande

```c
/* Logical Expressions 0.1 AC */
#include <stdio.h>

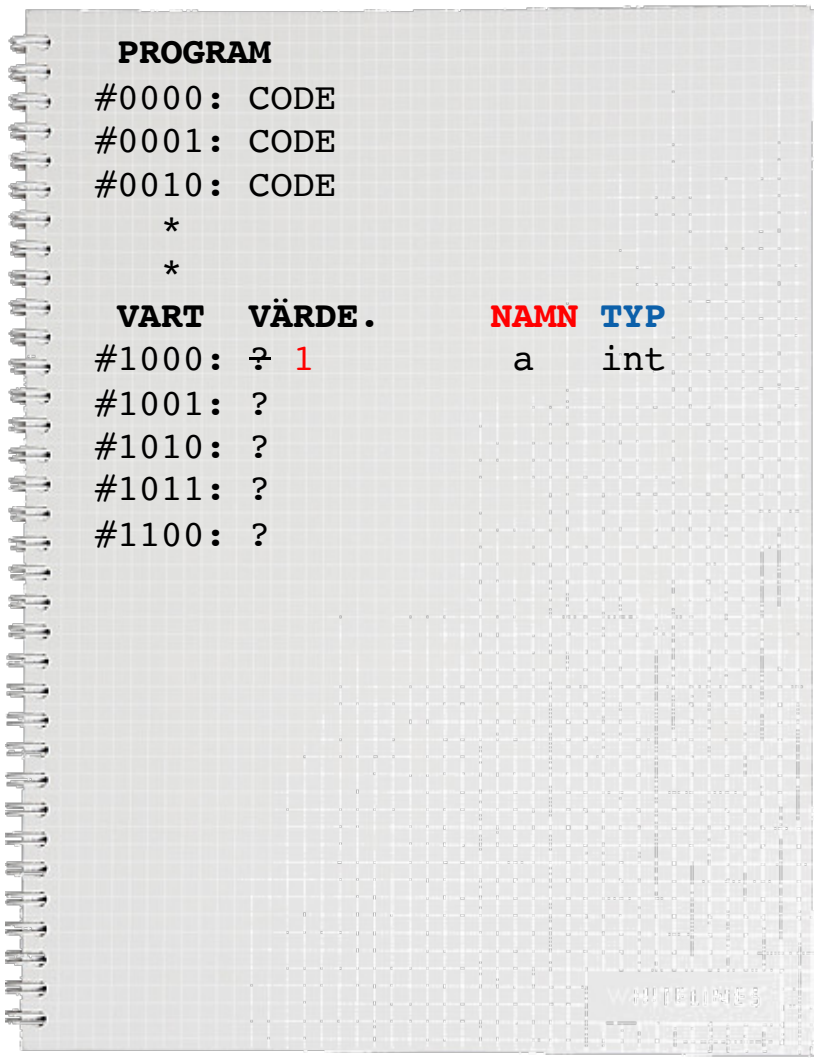
int main(void){
    int a;

    printf("Logical Expressions Tester (LET)!\n");

    printf("4==5 evaluates to: %d\n", 4==5);
    a = (4==4);
    printf("4==4 evaluates to: %d\n", a);

    printf("LET Done.\n");
    return 0;
}
```

```
Logical Expressions Tester (LET)!
4==5 evaluates to: 0
4==4 evaluates to: 1
LET Done.
```



(Det är mycket vanligt att utvecklare drar nytta av att alla heltal utom 0 är sant!)

Logiska

Ofta behövs flera jämförande uttryck kombineras…
…det kan ske med de logiska operationerna:
AND, OR & NOT.

Operator	Meaning of Operator	Example
&&	Logial AND	If c=5 and d=2 then,((c==5) && (d>5)) returns false.
||	Logical OR	If c=5 and d=2 then, ((c==5) || (d>5)) returns true.
!	Logical NOT	If c=5 then, !(c==5) returns false.

AND

```
F F : F   Result must
F T : F   be F skip!
T F : F
T T : T
```

OR

```
F F : F
F T : T
T F : T   Result must
T T : T   be T skip!
```

NOT

```
F   : T
T   : F
```


(Se upp med && och || som bara utför den första testen om utfallet är givet, short-circuit!)

Sammantaget

```c
/* Logical Expressions 0.2 AC */
#include <stdio.h>

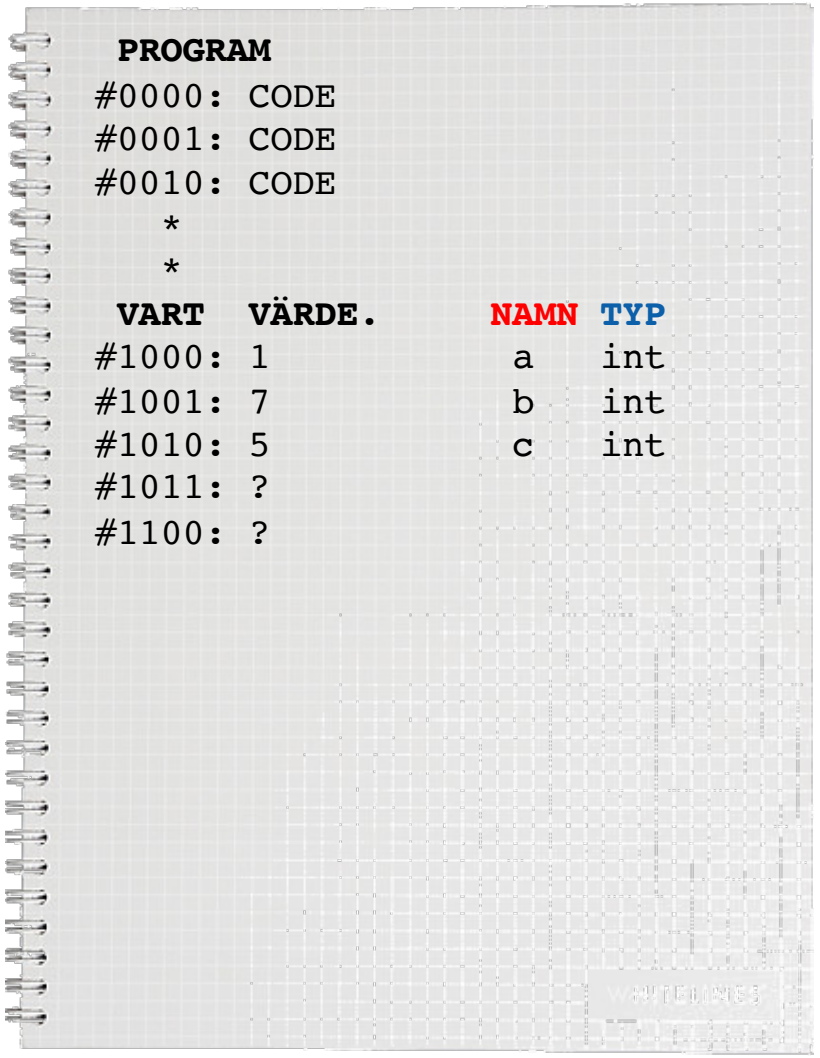
int main(void){
    int a=1, b=7, c=5;

    printf("Logical Expressions Tester (LET)!\n");

    printf("a>0 and b<9 : %d\n", a>0 && b<9);
    printf("a<b<c: %d\n", a<b<c);
    printf("a<b and b<c : %d\n", a<b && b<c);
    a<0 && ++b<9; /*Legal statement, value discarded */
    printf("a<0 and ++b<9 : b is %d\n", b);

    printf("LET Done.\n");
    return 0;
}
```

Inget konstigt…
…ge mig tre int variabler…
…alla med startvärden.



Sammantaget

```c
/* Logical Expressions 0.2 AC */
#include <stdio.h>

int main(void){
    int a=1, b=7, c=5;

    printf("Logical Expressions Tester (LET)!\n");

    printf("a>0 and b<9 : %d\n", a>0 && b<9);
    printf("a<b<c: %d\n", a<b<c);
    printf("a<b and b<c : %d\n", a<b && b<c);
    a<0 && ++b<9; /*Legal statement, value discarded */
    printf("a<0 and ++b<9 : b is %d\n", b);

    printf("LET Done.\n");
    return 0;
}
```

a>0 sant (1)…
b<9 sant (1)…
…sant && sant är sant (1), skriver ut 1!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.       NAMN  TYP
#1000: 1              a     int
#1001: 7              b     int
#1010: 5              c     int
#1011: ?
#1100: ?
```

Sammantaget

```c
/* Logical Expressions 0.2 AC */
#include <stdio.h>

int main(void){
    int a=1, b=7, c=5;

    printf("Logical Expressions Tester (LET)!\n");

    printf("a>0 and b<9 : %d\n", a>0 && b<9);
    printf("a<b<c: %d\n", a<b<c);
    printf("a<b and b<c : %d\n", a<b && b<c);
    a<0 && ++b<9; /*Legal statement, value discarded */
    printf("a<0 and ++b<9 : b is %d\n", b);

    printf("LET Done.\n");
    return 0;
}
```

Eh…

…a<b<c…

…verkar falskt?!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.      NAMN  TYP
#1000: 1            a     int
#1001: 7            b     int
#1010: 5            c     int
#1011: ?
#1100: ?
```

Sammantaget

```c
/* Logical Expressions 0.2 AC */
#include <stdio.h>

int main(void){
    int a=1, b=7, c=5;

    printf("Logical Expressions Tester (LET)!\n");

    printf("a>0 and b<9 : %d\n", a>0 && b<9);
    printf("a<b<c: %d\n", a<b<c);
    printf("a<b and b<c : %d\n", a<b && b<c);
    a<0 && ++b<9; /*Legal statement, value discarded */
    printf("a<0 and ++b<9 : b is %d\n", b);

    printf("LET Done.\n");
    return 0;
}
```

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.      NAMN  TYP
#1000: 1             a     int
#1001: 7             b     int
#1010: 5             c     int
#1011: ?
#1100: ?
```

3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ −	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	

Beräknas som (a<b)<c…

…a<b är sant (1)…

…ger 1<c vilket är sant (1), skriver ut 1!

Använd aldrig skrivsättet min<x<max, det fungerar inte!

Sammantaget

```c
/* Logical Expressions 0.2 AC */
#include <stdio.h>

int main(void){
    int a=1, b=7, c=5;

    printf("Logical Expressions Tester (LET)!\n");

    printf("a>0 and b<9 : %d\n", a>0 && b<9);
    printf("a<b<c: %d\n", a<b<c);
    printf("a<b and b<c : %d\n", a<b && b<c);
    a<0 && ++b<9; /*Legal statement, value discarded */
    printf("a<0 and ++b<9 : b is %d\n", b);

    printf("LET Done.\n");
    return 0;
}
```

a<b är sant (1)…
b<c är falskt (0)…
…sant && falskt är falskt, skriver ut 0!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.       NAMN  TYP
#1000: 1              a     int
#1001: 7              b     int
#1010: 5              c     int
#1011: ?
#1100: ?
```

Sammantaget

```c
/* Logical Expressions 0.2 AC */
#include <stdio.h>

int main(void){
    int a=1, b=7, c=5;

    printf("Logical Expressions Tester (LET)!\n");

    printf("a>0 and b<9 : %d\n", a>0 && b<9);
    printf("a<b<c: %d\n", a<b<c);
    printf("a<b and b<c : %d\n", a<b && b<c);
    a<0 && ++b<9; /*Legal statement, value discarded */
    printf("a<0 and ++b<9 : b is %d\n", b);

    printf("LET Done.\n");
    return 0;
}
```

Ett uttryck kan vara en sats, glöm inte ’;’
Värdet behöver inte användas!
Men det kan finnas sidoeffekter som består!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.      NAMN  TYP
#1000: 1             a     int
#1001: 7             b     int
#1010: 5             c     int
#1011: ?
#1100: ?
```


a<0 är falskt, så ++b<9 kommer inte att utföras (speciellt inte ++b), värdet är falskt (0) som kastas!

Sammantaget

```c
/* Logical Expressions 0.2 AC */
#include <stdio.h>

int main(void){
    int a=1, b=7, c=5;

    printf("Logical Expressions Tester (LET)!\n");

    printf("a>0 and b<9 : %d\n", a>0 && b<9);
    printf("a<b<c: %d\n", a<b<c);
    printf("a<b and b<c : %d\n", a<b && b<c);
    a<0 && ++b<9; /*Legal statement, value discarded */
    printf("a<0 and ++b<9 : b is %d\n", b);

    printf("LET Done.\n");
    return 0;
}
```

```
Logical Expressions Tester (LET)!
a>0 and b<9 : 1
a<b<c: 1
a<b and b<c : 0
a<0 and ++b<9 : b is 7
LET Done.
```

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.      NAMN  TYP
#1000: 1            a     int
#1001: 7            b     int
#1010: 5            c     int
#1011: ?
#1100: ?
```



Övning

Skapa ett program som hjälper dig analysera vilken livsform du observerar baserat på livsformens antal ben!

- 0-4 Däggdjur
- 5-7 Insekt
- 8 Spindel
- Fler Usch!

Tips: Använd printf(”xxxx : %d”, uttryck);

Vad är första steget i programutvecklingen?

```
My object analyzer.
Enter nb of legs : 2
Mammal! : 1
Insect! : 0
Spider! : 0
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 6
Mammal! : 0
Insect! : 1
Spider! : 0
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 8
Mammal! : 0
Insect! : 0
Spider! : 1
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 10
Mammal! : 0
Insect! : 0
Spider! : 0
Creepy! : 1
My analyzer done!
```



Övning

Skapa ett program som hjälper dig analysera vilken livsform du observerar baserat på livsformens antal ben!

- 0-4 Däggdjur
- 5-7 Insekt
- 8 Spindel
- Fler Usch!

1 Skriv ner proble-met så att ett barn förstår det!

// Välkomsthälsning

// Fråge efter antal ben?
// Läs in antal ben

// Däggdjur?
// Insekt?
// Spindel?
// Usch?

// Avslutningstext

```
My object analyzer.
Enter nb of legs : 2
Mammal! : 1
Insect! : 0
Spider! : 0
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 6
Mammal! : 0
Insect! : 1
Spider! : 0
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 8
Mammal! : 0
Insect! : 0
Spider! : 1
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 10
Mammal! : 0
Insect! : 0
Spider! : 0
Creepy! : 1
My analyzer done!
```

Övning

Skapa ett program som hjälper dig analysera vilken livsform du observerar baserat på livsformens antal ben!

- 0-4 Däggdjur
- 5-7 Insekt
- 8 Spindel
- Fler Usch!

2 **Gör ett minimalt main() av dina anteckningar!**

```
#include <stdio.h>

int main(void) {
  int legs;

  printf("My object analyzer.\n");

  printf("Enter nb of legs : ");
  // Läs in antal ben

  // Däggdjur?
  // Insekt?
  // Spindel?
  // Usch?

  printf("My analyzer done!\n");
  return 0;
}
```

```
My object analyzer.
Enter nb of legs : 2
Mammal! : 1
Insect! : 0
Spider! : 0
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 6
Mammal! : 0
                  1
                  0
                  0
                er done!
                :~ vc03$ /Use
                 analyzer.
                of legs : 8
                  0
                  0
                  1
                  0
                er done!
                :~ vc03$ /Use
                 analyzer.
                of legs : 10
                  0
                  0
                  0
                  1
                er done!
```

Övning

Skapa ett program som hjälper dig analysera vilken livsform du observerar baserat på livsformens antal ben!

- 0-4 Däggdjur
- 5-7 Insekt
- 8 Spindel
- Fler Usch!

Konstanter
Variabler med värde
Uttryck & resultat!

3a..d

```
#include <stdio.h>

int main(void) {
  int legs = 2;        = 5;        = 8;        = 100;

  printf("My object analyzer.\n");

  printf("Enter nb of legs : ");
  // Läs in antal ben

  printf("Mammal! : %d\n", legs<=4);
  printf("Insect! : %d\n", ((legs>4) && (legs<8)));
  printf("Spider! : %d\n", legs==8);
  printf("Creepy! : %d\n",legs>8);

  printf("My analyzer done!\n");
  return 0;
}
```

```
My object analyzer.
Enter nb of legs : 2
Mammal! : 1
Insect! : 0
Spider! : 0
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 6
Mammal! : 0
: 1
: 0
: 0
zer done!
6:~ vc03$ /Use
t analyzer.
 of legs : 8
: 0
: 0
: 1
: 0
zer done!
6:~ vc03$ /Use
t analyzer.
 of legs : 10
: 0
: 0
: 0
: 1
zer done!
```

Övning

Skapa ett program som hjälper dig analysera vilken livsform du observerar baserat på livsformens antal ben!

- 0-4 Däggdjur
- 5-7 Insekt
- 8 Spindel
- Fler Usch!

4 **Läs in riktiga värden!**

```c
#include <stdio.h>

int main(void) {
  int legs;

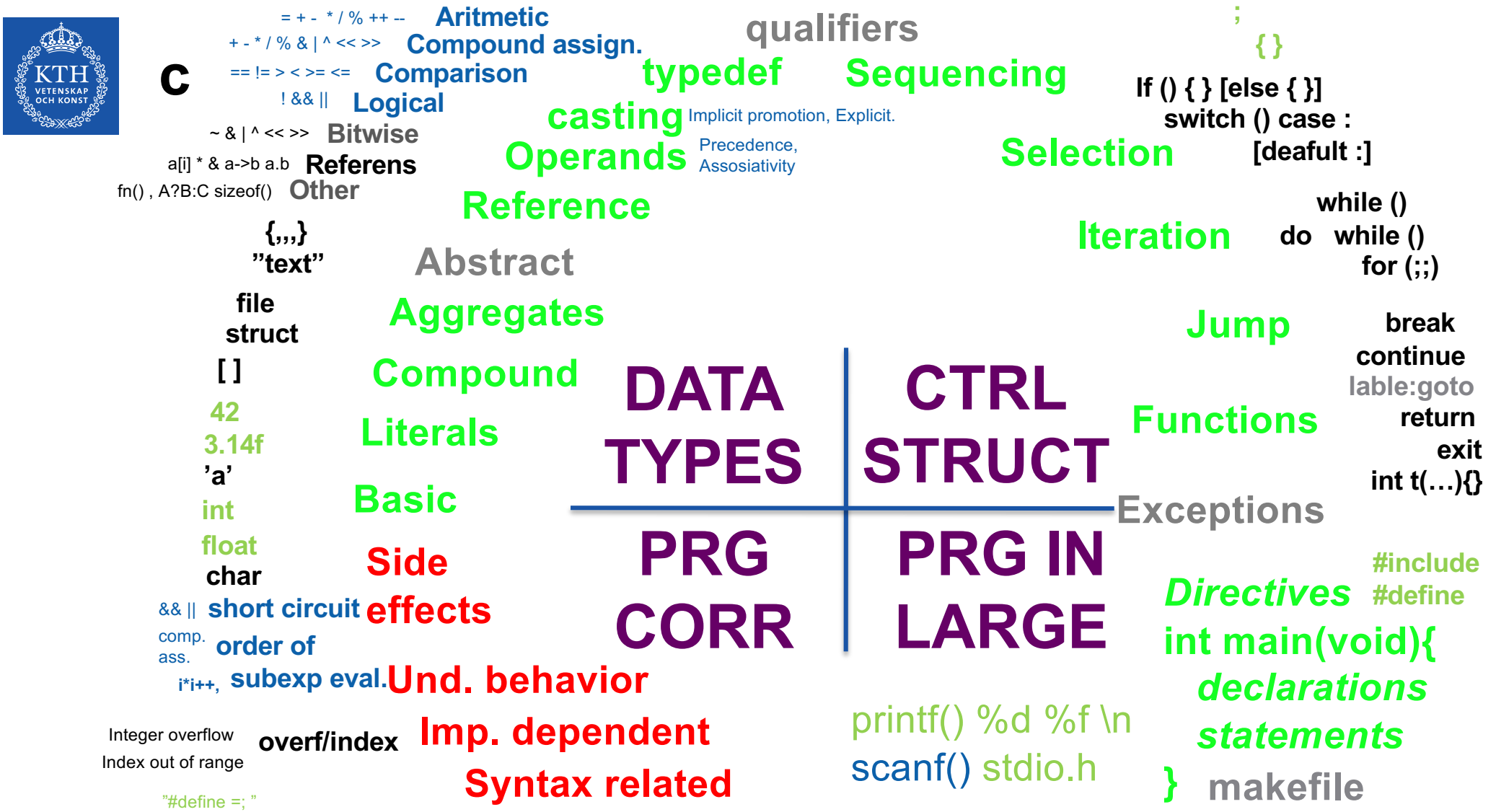
  printf("My object analyzer.\n");

  printf("Enter nb of legs : ");
  scanf("%d", &legs);

  printf("Mammal! : %d\n", legs<=4);
  printf("Insect! : %d\n", ((legs>4) && (legs<8)));
  printf("Spider! : %d\n", legs==8);
  printf("Creepy! : %d\n",legs>8);

  printf("My analyzer done!\n");
  return 0;
}
```

```
My object analyzer.
Enter nb of legs : 2
Mammal! : 1
Insect! : 0
Spider! : 0
Creepy! : 0
My analyzer done!
n136-p186:~ vc03$ /Use
My object analyzer.
Enter nb of legs : 6
Mammal! : 0
: 1
: 0
: 0
zer done!
6:~ vc03$ /Use
t analyzer.
 of legs : 8
: 0
: 0
: 1
: 0
zer done!
6:~ vc03$ /Use
t analyzer.
 of legs : 10
: 0
: 0
: 0
: 1
zer done!
```


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass. order of
i*i++, subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
Integer overflow
Index out of range
overf/index
”#define =; ”
printf() %d %f \n
scanf() stdio.h
int/int
/*comment*/ or //

Övningar #2: Kapitel 3:P1,P6 & P5

MM/DD/YYYY
To
YYYYMMDD

16 3 2 13
5 10 11 8
9 6 7 12
4 **15** **14** 1

5/6*3/4
To
15/24

34 34 34 34
34 34 34 34
34 34

16 3 2 13 5 10 11 8 9 6 7 12 4 15 14 1

1 Skriv ner problemet så att ett barn förstår det!

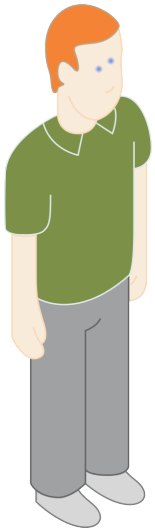
2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter Variabler med Värde Uttryck & resultat!

a..n

4 Läs in riktiga värden!

Historik



• 150520 0.1	AC	Skapad.
• 150626 0.5	AC	Uttryck & operationer.
• 150630 0.9	AC	Övningsexempel.
• 150821 1.0	AC	Föreläsningsövningsuppgifter.
• 160808 1.1	AC	Mindre uppdateringar.
• 170730 1.2	AC	Metodiken bättre betonad.
• 190730 2.0	AC	Omarbetad från grunden, samt anpassad till KTH:s nya profil.